

Lacuna · *One at a time*

The first accessibility tool built for the *hearing* side of the table. Every captioning app gives the deaf or hard-of-hearing (DHH) person more to read. We built the other side: every phone on the table becomes a lamp that turns amber when two people talk at once or someone talks too fast, while the DHH person's own phone shows one sentence at a time — name, tone, and a live decision ledger — with nothing ever recorded to disk.

Stack

CLIENT Vite + React + TypeScript + Tailwind v4, built as a phone-first web app with a PWA manifest — no app-store install, scan a QR and it opens.

SERVER Fastify + `ws` on Node, shared TypeScript types imported by both sides so the wire contract can't drift.

DEPLOY Live at `https://table.akilion.ai` through a named Cloudflare tunnel (`imt-table`) — chosen because venue Wi-Fi sometimes blocks the quick `*.trycloudflare.com` hostnames a hackathon demo can't risk.

COST About **\$6** per hour of live table (Deepgram + Claude combined).

Tables: the unit of a session

Each listener's phone creates a **table** identified by a 4-letter code drawn from an alphabet that drops characters people misread aloud (no `0/0` , `1/I/L`). Every other phone joining types that code; every HTTP call and every WebSocket connection after that is scoped by it (`server/src/rooms.ts` , `createRoom` / `findRoom`). Tables live in server memory only — there is no database of active sessions — so a code re-opens transparently after a server restart: a phone holding a stale code lands on a freshly created room with the same code rather than seeing "offline".

Two operating modes

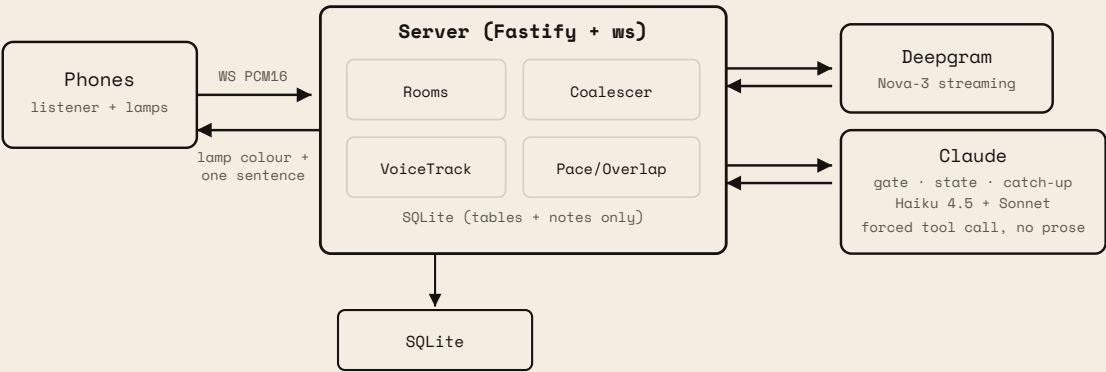
One phone, many voices

A single phone sits in the middle. Deepgram's own diarization labels turns "Speaker 1", "Speaker 2" — tap a label once to rename it. Optionally, each person enrolls a 6-second voice sample; a local speaker-ID model then names lines by *voice*, not by Deepgram's turn-taking guess, and survives Deepgram relabelling people mid-conversation.

Multi-phone ("everyone joins")

Each hearing person opens the join link on their own phone and speaks into their own mic. Their phone is simultaneously a source of ground-truth-attributed transcript and a "lamp" — no diarization guessing needed because the phone *is* the identity.

Architecture



Audio and transcripts are never written to disk – only table metadata and ledger notes persist.

Audio path, mic to speaker identity

Every mic (host, phone, or enrollment clip) runs the same cleanup chain (`client/src/audio/clean.ts`) so Deepgram and the voice-ID model always see identically conditioned audio:

```
getUserMedia (echo cancel · noise suppression · AGC · voice isolation) → high-pass 90 Hz (table thumps, handling noise, HVAC) → +2 dB peaking presence @ 3 kHz (consonant clarity) → dynamics compressor (threshold -28 dB, ratio 3:1) → make-up gain → AudioWorklet downsample to 16 kHz PCM16 → WebSocket → server → Deepgram Nova-3 streaming
```

Deepgram is configured with diarization, interim results, 400 ms endpointing (the default 10 ms endpointing over-fragmented turns — 41 finals for 23 scripted lines in early testing), and keyterm boosts seeded with the listener's name, the table's names, and proper nouns pulled from the live ledger. Languages: English, Italian, Turkish.

Speaker identity: two paths

One phone mode — voice prints

Deepgram's diarization gives raw "Speaker 1, 2..." ids, immediately usable by tapping to rename. Layered on top, an optional voice-print system removes the need to babysit Deepgram's turn attribution:

- **Engine:** sherpa-onnx-node (ONNX Runtime, native, no Python) running NeMo TitaNet-small speaker embeddings on-CPU.
- **Enrollment:** 6 seconds of voiced audio per person, passed around the phone once.
- **Live buffer:** a 30-second PCM ring per socket; every 400 ms the last 1.2 s is re-embedded and compared against enrolled centroids.
- **Decision rule:** relative, not absolute — the best match must score ≥ 0.45 and be ≥ 0.15 ahead of the runner-up before a line is named. A confidently-named live line becomes a new enrollment sample (live adaptation), so the print tracks the room and the mic over the first minute.
- **A speaker change inside one Deepgram turn** splits the caption line — the voice fingerprint decides the turn boundary, Deepgram's diarization id is only a hint (server/src/voicetrack.ts).

Measured: 10–35 ms per identify call on an M-series CPU. Synthetic TTS voices: 27/29 lines correctly named on one mic. Enrolled clips: 100% correct. Unenrolled strangers: 83% correctly rejected as "unknown". **Real phone-mic voices score much lower than synthetic speech** — cosine similarity 0.2–0.6 instead of TTS's 0.76–0.92 — which is exactly why unresolved speakers default to a plain "Speaker N" instead of a guessed name: a wrong name is worse than no name.

Multi-phone mode — lamps, not fingerprints

When every hearing person's phone is its own mic, speaker identity is free (it's their phone) and the phone becomes a "lamp" instead:

- **Green** — Go ahead.
- **Amber** — "One at a time" on detected overlap with another phone (per-phone, computed server-side from simultaneous voiced 100 ms slots across sources), or "Slower" once speaking rate crosses 185 wpm (250 wpm on the same syllable-based scale for the fast/too-fast split at 210 wpm).
- **Cross-phone bleed** (a loud neighbour's voice leaking into a quieter phone's mic) is removed by comparing each word's loudness against that phone's own speaking-level baseline versus how "owned" it looks on the source that's actually talking.

Measured: 24/24 lines correctly attributed to the right speaker under simulated overlap and cross-phone bleed.

The Claude layer — three calls, one contract

Every model call is a forced tool call or a tightly constrained enum reply — never free prose — so latency and cost stay bounded and the client never has to parse Claude out of a paragraph.

CALL	MODEL	TRIGGER	OUTPUT	MEASURED LATENCY
Gate	Haiku 4.5	every finalized ASR line	~5-token verdict: addressed-to-me, confidence, kind, urgent, tone	~0.8 s p50
State / ledger	Haiku	rolling, every ~8 s (90 s window)	delta ledger updates, plans with reasons ≤ 6 words, conversation threads, table mood	~3 s
Catch-up	Sonnet	on demand ("what did I miss?")	≤ 3 bullet lines	~3–4 s

The gate also drafts "interject" suggestions inline. All calls go through an Anthropic workspace header for usage attribution.

Data flow

STAGE	FROM → TO	PAYLOAD	NOTES
1. Capture	Phone mic → cleanChain	raw audio	echo cancel, noise suppression, AGC, voice isolation
2. Encode	AudioWorklet → WebSocket	16 kHz PCM16	downsampled in-browser, no server-side resampling
3. Transcribe	Server → Deepgram Nova-3	PCM16 stream	diarize, interim + final results, endpointing 400 ms, keyterm boosts
4. Identify	PcmRing → VoiceTrack	1.2 s embedding window	one-phone mode only; every 400 ms
5. Coalesce	Rooms / Coalescer	finalized Utterance	merges fragments, dedupes cross-phone bleed
6. Classify	Server → Claude gate	target line + recent context	Haiku, forced 5-token reply
7. Render	Server → phones (WS)	Utterance / lamp state / mood	listener gets sentence+tone; lamps get colour+word
8. Persist	Server → SQLite	table row + notes	only metadata and ledger text — never audio or transcript

Latency budget

PATH	TYPICAL	NOTES
Mic → Deepgram interim	sub-second, streaming	interim results shown live, not final
Deepgram endpointing → final	400 ms	tuned up from Deepgram's 10 ms default to avoid over-fragmenting turns
Voice identify (one-phone mode)	10–35 ms	per 1.2 s window, on-CPU
Gate (Haiku, per line)	~0.8 s p50	5-token forced reply
State / ledger refresh	~3 s, every ~8 s	90 s rolling window
Catch-up ("what did I miss?")	~3–4 s	Sonnet, ≤ 3 lines
Lamp pace / overlap update	every 2 s	server-computed, no model call

Listener UI, in one line each

- One sentence at a time, name and tone word — nothing to scroll, nothing to miss mid-read.
- Amber full-screen "asked you" moment with YES CLARIFY CAN'T, broadcast as text and speech to every phone at the table.
- "Say something" — types on the listener's phone, reads aloud and displays as text on every other phone.
- Conversation map — an SVG force layout of topics, people, and mood, built from the same threads the ledger tracks.
- Places — Leaflet + CARTO tiles, one pin per table that's been used.

Persistence

SQLite via Node's built-in `node:sqlite` — two tables only: `tables(id, code, listener, location, lat, lng, start, end)` and `notes(kind, text, speaker, t, reason)`. **Audio and transcripts are never stored** — only table metadata (who, where, when) and the notes the ledger produced (plans, pushback, catch-ups) persist past the live session.

Honest limits

- **One microphone cannot separate two people talking at once.** That physical fact is exactly why the lamp exists in multi-phone mode instead of trying to out-engineer diarization on overlapping speech.
- **Loud rooms degrade diarization** in one-phone mode — background noise and cross-talk push Deepgram's speaker boundaries and the voice-print confidence down together.
- **Voice prints need tuning on real voices.** The relative decision rule (0.45 floor, 0.15 margin) was tuned against real phone-mic recordings precisely because TTS voices score far higher (0.76–0.92) than real speech from across a table (0.2–0.6) — a threshold tuned on synthetic voices alone would over-trust itself in the real world.

Cost

Roughly **\$6** per hour of live table, combining Deepgram Nova-3 streaming and the three-tier Claude call pattern (Haiku gate + Haiku state + Sonnet catch-up).

Why this shape

Every design choice traces back to one constraint: nothing about a family dinner or a work standup should be recorded. That rules out a server-side transcript store, which rules out "search your meeting later," which is fine — the product is live decision support, not an archive. It also means the Claude calls are structural (typed tool calls on a rolling in-memory window) rather than conversational, so the model never accumulates a persistent transcript to reason over — each call gets exactly the window it needs and nothing survives it except the ledger's own short text.

One at a time — BAINSA hackathon 2026, topic "I Missed That". Live: [table.akilion.ai](#) · Repo: [github.com/laqostine/one-at-a-time](#) · Design: [design/DESIGN.md](#)